INVERTIS UNIVERSITY, BAREILLY

B.Tech CSE – Third Year

DBMS & SQL Lab (BCS552)

LAB PRACTICAL RECORD – 15 EXPERIMENTS

Student Name	**Ayush Kumar**
Student ID	**BCS2024552**
Roll Number	**2410303138**
Section / Batch	**D**

Pract. No.	Practical Name
01	Creating a Table and Adding Rows
02	Updating and Deleting Rows
03	Selecting Columns and Rows with WHERE
04	Combining Conditions
05	Pattern Matching with LIKE
06	Sorting Rows and Removing Repeats
07	Using Aggregate Functions
08	Grouping Rows and Using HAVING
09	Aliases and CASE
10	Joining Two Tables with INNER JOIN
11	Other Kinds of Joins
12	Using a Subquery
13	UNION and INTERSECT
14	Creating and Using a View
15	Indexes and Adding a Row

EXPERIMENT NO. 1

1. Experiment Title

Creating a Table and Adding Rows

2. Aim

Create a table, add three rows with one INSERT, and show the table.

3. Software / Tools Required

MySQL or another compatible SQL database system, SQL editor/command-line client, and a tool to capture output screenshots.

4. Theory

This experiment practices the SQL concepts named in the experiment title. SQL statements are used to define database objects, manipulate records, retrieve data, and display query results according to the task requirements.

5. Algorithm / Procedure

1. Open the SQL editor and select the appropriate database.
2. Run the setup statements below, if provided, to create and populate the required tables.
3. Execute each task query in order.
4. Check the returned rows or database changes against the task requirements.
5. Capture the actual output for the practical record.

6. Queries

1. Create the student table

```
CREATE TABLE student (
    roll_no INT,
    name VARCHAR(30),
    marks INT
);
```

2. Insert three rows in one statement

```
INSERT INTO student (roll_no, name, marks)
VALUES
    (101, 'Asha', 78),
    (102, 'Rohan', 64),
    (103, 'Neha', 88);
```

3. Show all rows and columns

```
SELECT * FROM student;
```

7. Result

The SQL queries were executed and the results were checked against the requirements of the experiment.

8. Viva-Voce Questions

1. What is the purpose of CREATE TABLE?
2. What is the difference between INT and VARCHAR?
3. How can multiple rows be inserted using one INSERT statement?

EXPERIMENT NO. 2

1. Experiment Title

Updating and Deleting Rows

2. Aim

Change values in rows, then delete a row and check the table after each step.

3. Software / Tools Required

MySQL or another compatible SQL database system, SQL editor/command-line client, and a tool to capture output screenshots.

4. Theory

This experiment practices the SQL concepts named in the experiment title. SQL statements are used to define database objects, manipulate records, retrieve data, and display query results according to the task requirements.

5. Algorithm / Procedure

1. Open the SQL editor and select the appropriate database.
2. Run the setup statements below, if provided, to create and populate the required tables.
3. Execute each task query in order.
4. Check the returned rows or database changes against the task requirements.
5. Capture the actual output for the practical record.

6. Queries

Setup (run first; not a task)

```
CREATE TABLE stock (item_code INT, item_name VARCHAR(30), quantity INT);
INSERT INTO stock (item_code, item_name, quantity) VALUES (1, 'Notebook', 50), (2, 'Pen', 120),
(3, 'Bag', 0);
```

1. Set quantity of item_code 1 to 80

```
UPDATE stock SET quantity = 80 WHERE item_code = 1;
SELECT * FROM stock;
```

2. Add 10 to quantity of item_code 2

```
UPDATE stock SET quantity = quantity + 10 WHERE item_code = 2;
SELECT * FROM stock;
```

3. Delete item_code 3

```
DELETE FROM stock WHERE item_code = 3;
SELECT * FROM stock;
```

7. Result

The SQL queries were executed and the results were checked against the requirements of the experiment.

8. Viva-Voce Questions

1. What is the purpose of UPDATE?
2. Why is WHERE important in UPDATE and DELETE statements?
3. What happens if DELETE is executed without a WHERE clause?

EXPERIMENT NO. 3

1. Experiment Title

Selecting Columns and Rows with WHERE

2. Aim

Show selected columns and rows matching conditions.

3. Software / Tools Required

MySQL or another compatible SQL database system, SQL editor/command-line client, and a tool to capture output screenshots.

4. Theory

This experiment practices the SQL concepts named in the experiment title. SQL statements are used to define database objects, manipulate records, retrieve data, and display query results according to the task requirements.

5. Algorithm / Procedure

1. Open the SQL editor and select the appropriate database.
2. Run the setup statements below, if provided, to create and populate the required tables.
3. Execute each task query in order.
4. Check the returned rows or database changes against the task requirements.
5. Capture the actual output for the practical record.

6. Queries

Setup (run first; not a task)

```
CREATE TABLE student (roll_no INT, name VARCHAR(30), marks INT);
INSERT INTO student (roll_no, name, marks) VALUES (101, 'Asha', 78), (102, 'Rohan', 64), (103,
'Neha', 88), (104, 'Deepak', 52);
```

1. Show name and marks of all students

```
SELECT name, marks FROM student;
```

2. Show students with marks greater than 60

```
SELECT * FROM student WHERE marks > 60;
```

3. Show the name of roll number 103

```
SELECT name FROM student WHERE roll_no = 103;
```

7. Result

The SQL queries were executed and the results were checked against the requirements of the experiment.

8. Viva-Voce Questions

1. What is the purpose of SELECT?
2. How does WHERE filter rows?
3. What is the difference between SELECT * and selecting specific columns?

EXPERIMENT NO. 4

1. Experiment Title

Combining Conditions

2. Aim

Combine conditions with AND, use BETWEEN for a range, and IN for a list.

3. Software / Tools Required

MySQL or another compatible SQL database system, SQL editor/command-line client, and a tool to capture output screenshots.

4. Theory

This experiment practices the SQL concepts named in the experiment title. SQL statements are used to define database objects, manipulate records, retrieve data, and display query results according to the task requirements.

5. Algorithm / Procedure

1. Open the SQL editor and select the appropriate database.
2. Run the setup statements below, if provided, to create and populate the required tables.
3. Execute each task query in order.
4. Check the returned rows or database changes against the task requirements.
5. Capture the actual output for the practical record.

6. Queries

Setup (run first; not a task)

```
CREATE TABLE player (name VARCHAR(30), team VARCHAR(20), runs INT);
INSERT INTO player (name, team, runs) VALUES ('Asha', 'Eagles', 45), ('Meera', 'Tigers', 72),
('Arjun', 'Eagles', 30), ('Kiran', 'Lions', 88);
```

1. Eagles players scoring more than 35 runs (AND)

```
SELECT name, runs FROM player WHERE team = 'Eagles' AND runs > 35;
```

2. Players with runs between 40 and 80

```
SELECT name, runs FROM player WHERE runs BETWEEN 40 AND 80;
```

3. Players in Tigers or Lions (IN)

```
SELECT name, team FROM player WHERE team IN ('Tigers', 'Lions');.
```

7. Result

The SQL queries were executed and the results were checked against the requirements of the experiment.

8. Viva-Voce Questions

1. What is the difference between AND and OR?
2. What does BETWEEN do?
3. How is IN used in a WHERE clause?

EXPERIMENT NO. 5

1. Experiment Title

Pattern Matching with LIKE

2. Aim

Find titles that match a pattern with LIKE.

3. Software / Tools Required

MySQL or another compatible SQL database system, SQL editor/command-line client, and a tool to capture output screenshots.

4. Theory

This experiment practices the SQL concepts named in the experiment title. SQL statements are used to define database objects, manipulate records, retrieve data, and display query results according to the task requirements.

5. Algorithm / Procedure

1. Open the SQL editor and select the appropriate database.
2. Run the setup statements below, if provided, to create and populate the required tables.
3. Execute each task query in order.
4. Check the returned rows or database changes against the task requirements.
5. Capture the actual output for the practical record.

6. Queries

Setup (run first; not a task)

```
CREATE TABLE book (title VARCHAR(40), category VARCHAR(20), copies INT);
INSERT INTO book (title, category, copies) VALUES ('Database Systems', 'Computer', 3), ('Digital
Electronics', 'Electronics', 2), ('Data Structures in C', 'Computer', 2), ('Short Stories',
'Literature', 1);
```

1. Titles starting with Data

```
SELECT title FROM book WHERE title LIKE 'Data%';
```

2. Titles ending with the letter C

```
SELECT title FROM book WHERE title LIKE '%C';
```

3. Titles whose second letter is h

```
SELECT title FROM book WHERE title LIKE '_h%';
```

7. Result

The SQL queries were executed and the results were checked against the requirements of the experiment.

8. Viva-Voce Questions

1. What is pattern matching in SQL?
2. What do % and _ mean in LIKE patterns?
3. How can you find values ending with a specific character?

EXPERIMENT NO. 6

1. Experiment Title

Sorting Rows and Removing Repeats

2. Aim

Sort rows, remove repeated values, and show only some rows.

3. Software / Tools Required

MySQL or another compatible SQL database system, SQL editor/command-line client, and a tool to capture output screenshots.

4. Theory

This experiment practices the SQL concepts named in the experiment title. SQL statements are used to define database objects, manipulate records, retrieve data, and display query results according to the task requirements.

5. Algorithm / Procedure

1. Open the SQL editor and select the appropriate database.
2. Run the setup statements below, if provided, to create and populate the required tables.
3. Execute each task query in order.
4. Check the returned rows or database changes against the task requirements.
5. Capture the actual output for the practical record.

6. Queries

Setup (run first; not a task)

```
CREATE TABLE product (product_name VARCHAR(30), category VARCHAR(20), price INT);
INSERT INTO product (product_name, category, price) VALUES ('Pen', 'Stationery', 15),
('Notebook', 'Stationery', 60), ('Calculator', 'Electronics', 850), ('Bag', 'Office', 500);
```

1. Products sorted by price, highest to lowest

```
SELECT product_name, price FROM product ORDER BY price DESC;
```

2. Each category once, sorted by category

```
SELECT DISTINCT category FROM product ORDER BY category ASC;
```

3. Two most expensive products

```
SELECT product_name, price FROM product ORDER BY price DESC LIMIT 2;
```

7. Result

The SQL queries were executed and the results were checked against the requirements of the experiment.

8. Viva-Voce Questions

1. What does ORDER BY do?
2. What is the purpose of DISTINCT?
3. How does LIMIT restrict returned rows?

EXPERIMENT NO. 7

1. Experiment Title

Using Aggregate Functions

2. Aim

Use an aggregate function in each task.

3. Software / Tools Required

MySQL or another compatible SQL database system, SQL editor/command-line client, and a tool to capture output screenshots.

4. Theory

This experiment practices the SQL concepts named in the experiment title. SQL statements are used to define database objects, manipulate records, retrieve data, and display query results according to the task requirements.

5. Algorithm / Procedure

1. Open the SQL editor and select the appropriate database.
2. Run the setup statements below, if provided, to create and populate the required tables.
3. Execute each task query in order.
4. Check the returned rows or database changes against the task requirements.
5. Capture the actual output for the practical record.

6. Queries

Setup (run first; not a task)

```
CREATE TABLE sales (salesperson VARCHAR(20), region VARCHAR(20), amount INT);
INSERT INTO sales (salesperson, region, amount) VALUES ('Deepak', 'North', 12000), ('Pooja',
'South', 8500), ('Deepak', 'South', 9500), ('Lata', 'North', 11000);
```

1. Count all sales rows

```
SELECT COUNT(*) AS num_sales FROM sales;
```

2. Total of all amounts

```
SELECT SUM(amount) AS total FROM sales;
```

3. Highest amount

```
SELECT MAX(amount) AS highest FROM sales;
```

7. Result

The SQL queries were executed and the results were checked against the requirements of the experiment.

8. Viva-Voce Questions

1. What are aggregate functions?
2. What is the difference between COUNT and SUM?
3. What does MAX return?

EXPERIMENT NO. 8

1. Experiment Title

Grouping Rows and Using HAVING

2. Aim

Group rows by region and check totals for each group.

3. Software / Tools Required

MySQL or another compatible SQL database system, SQL editor/command-line client, and a tool to capture output screenshots.

4. Theory

This experiment practices the SQL concepts named in the experiment title. SQL statements are used to define database objects, manipulate records, retrieve data, and display query results according to the task requirements.

5. Algorithm / Procedure

1. Open the SQL editor and select the appropriate database.
2. Run the setup statements below, if provided, to create and populate the required tables.
3. Execute each task query in order.
4. Check the returned rows or database changes against the task requirements.
5. Capture the actual output for the practical record.

6. Queries

Setup (run first; not a task)

```
CREATE TABLE shop_sale (salesperson VARCHAR(20), region VARCHAR(20), amount INT);
INSERT INTO shop_sale (salesperson, region, amount) VALUES ('Deepak', 'North', 12000), ('Pooja',
'South', 8500), ('Deepak', 'South', 9500), ('Lata', 'North', 11000);
```

1. Count sales in each region

```
SELECT region, COUNT(*) AS num_sales FROM shop_sale GROUP BY region ORDER BY region;
```

2. Total amount in each region

```
SELECT region, SUM(amount) AS total FROM shop_sale GROUP BY region ORDER BY region;
```

3. Regions with total amount greater than 20000

```
SELECT region, SUM(amount) AS total FROM shop_sale GROUP BY region HAVING SUM(amount) > 20000
ORDER BY region;
```

7. Result

The SQL queries were executed and the results were checked against the requirements of the experiment.

8. Viva-Voce Questions

1. What is the purpose of GROUP BY?
2. How does HAVING differ from WHERE?
3. Why is ORDER BY used after grouping?

EXPERIMENT NO. 9

1. Experiment Title

Aliases and CASE

2. Aim

Give columns new names and show a value chosen by a condition.

3. Software / Tools Required

MySQL or another compatible SQL database system, SQL editor/command-line client, and a tool to capture output screenshots.

4. Theory

This experiment practices the SQL concepts named in the experiment title. SQL statements are used to define database objects, manipulate records, retrieve data, and display query results according to the task requirements.

5. Algorithm / Procedure

1. Open the SQL editor and select the appropriate database.
2. Run the setup statements below, if provided, to create and populate the required tables.
3. Execute each task query in order.
4. Check the returned rows or database changes against the task requirements.
5. Capture the actual output for the practical record.

6. Queries

Setup (run first; not a task)

```
CREATE TABLE exam_result (name VARCHAR(20), marks INT);
INSERT INTO exam_result (name, marks) VALUES ('Asha', 92), ('Rohan', 47), ('Neha', 68), ('Meera',
33);
```

1. Rename output columns using AS

```
SELECT name AS student_name, marks AS score FROM exam_result;
```

2. Show Pass or Fail using CASE

```
SELECT name, CASE WHEN marks >= 40 THEN 'Pass' ELSE 'Fail' END AS result FROM exam_result;
```

3. Add five bonus marks

```
SELECT name, marks + 5 AS with_bonus FROM exam_result;
```

7. Result

The SQL queries were executed and the results were checked against the requirements of the experiment.

8. Viva-Voce Questions

1. What is a column alias?
2. What does a CASE expression do?
3. Can arithmetic expressions be used in SELECT?

EXPERIMENT NO. 10

1. Experiment Title

Joining Two Tables with INNER JOIN

2. Aim

Join department and employee tables on dept_id.

3. Software / Tools Required

MySQL or another compatible SQL database system, SQL editor/command-line client, and a tool to capture output screenshots.

4. Theory

This experiment practices the SQL concepts named in the experiment title. SQL statements are used to define database objects, manipulate records, retrieve data, and display query results according to the task requirements.

5. Algorithm / Procedure

1. Open the SQL editor and select the appropriate database.
2. Run the setup statements below, if provided, to create and populate the required tables.
3. Execute each task query in order.
4. Check the returned rows or database changes against the task requirements.
5. Capture the actual output for the practical record.

6. Queries

Setup (run first; not a task)

```
CREATE TABLE department (dept_id INT, dept_name VARCHAR(20));
INSERT INTO department (dept_id, dept_name) VALUES (10, 'Accounts'), (20, 'Sales'), (40,
'Research');
CREATE TABLE employee (emp_name VARCHAR(30), dept_id INT, salary INT);
INSERT INTO employee (emp_name, dept_id, salary) VALUES ('Anita', 10, 90000), ('Lata', 20,
55000), ('Deepak', 10, 60000), ('Pooja', 20, 42000);
```

1. Employee names with department names

```
SELECT employee.emp_name, department.dept_name FROM employee INNER JOIN department ON
employee.dept_id = department.dept_id ORDER BY employee.emp_name;
```

2. Employees in Sales

```
SELECT employee.emp_name FROM employee INNER JOIN department ON employee.dept_id =
department.dept_id WHERE department.dept_name = 'Sales' ORDER BY employee.emp_name;
```

3. Employee, department, and salary (highest first)

```
SELECT employee.emp_name, department.dept_name, employee.salary FROM employee INNER JOIN
department ON employee.dept_id = department.dept_id ORDER BY employee.salary DESC;
```

7. Result

The SQL queries were executed and the results were checked against the requirements of the experiment.

8. Viva-Voce Questions

1. What is an INNER JOIN?
2. Why is a join condition required?
3. How can joined results be sorted?

EXPERIMENT NO. 11

1. Experiment Title

Other Kinds of Joins

2. Aim

Use LEFT JOIN, CROSS JOIN, and NATURAL JOIN on two small tables.

3. Software / Tools Required

MySQL or another compatible SQL database system, SQL editor/command-line client, and a tool to capture output screenshots.

4. Theory

This experiment practices the SQL concepts named in the experiment title. SQL statements are used to define database objects, manipulate records, retrieve data, and display query results according to the task requirements.

5. Algorithm / Procedure

1. Open the SQL editor and select the appropriate database.
2. Run the setup statements below, if provided, to create and populate the required tables.
3. Execute each task query in order.
4. Check the returned rows or database changes against the task requirements.
5. Capture the actual output for the practical record.

6. Queries

Setup (run first; not a task)

```
CREATE TABLE team (team_id INT, team_name VARCHAR(20));
INSERT INTO team (team_id, team_name) VALUES (1, 'Eagles'), (2, 'Tigers'), (3, 'Lions');
CREATE TABLE player (player_name VARCHAR(30), team_id INT, age INT);
INSERT INTO player (player_name, team_id, age) VALUES ('Asha', 1, 24), ('Meera', 1, 22),
('Arjun', 2, 27);
```

1. Teams and their players, including teams with no players

```
SELECT team.team_name, player.player_name FROM team LEFT JOIN player ON team.team_id =
player.team_id ORDER BY team.team_name, player.player_name;
```

2. Every pair of team and player names

```
SELECT team.team_name, player.player_name FROM team  CROSS JOIN player ORDER BY team.team_name,
player.player_name;
```

3. Each player with their team name (NATURAL JOIN)

```
SELECT player_name, team_name FROM player NATURAL JOIN team ORDER BY player_name;
```

7. Result

The SQL queries were executed and the results were checked against the requirements of the experiment.

8. Viva-Voce Questions

6. How does LEFT JOIN differ from INNER JOIN?
7. What does CROSS JOIN return?
8. What is a NATURAL JOIN based on?

EXPERIMENT NO. 12

1. Experiment Title

Using a Subquery

2. Aim

Use a query inside another query, with IN.

3. Software / Tools Required

MySQL or another compatible SQL database system, SQL editor/command-line client, and a tool to capture output screenshots.

4. Theory

This experiment practices the SQL concepts named in the experiment title. SQL statements are used to define database objects, manipulate records, retrieve data, and display query results according to the task requirements.

5. Algorithm / Procedure

1. Open the SQL editor and select the appropriate database.
2. Run the setup statements below, if provided, to create and populate the required tables.
3. Execute each task query in order.
4. Check the returned rows or database changes against the task requirements.
5. Capture the actual output for the practical record.

6. Queries

Setup (run first; not a task)

```
CREATE TABLE club_a (student_name VARCHAR(30));
INSERT INTO club_a (student_name) VALUES ('Asha'), ('Rohan'), ('Neha');
CREATE TABLE club_b (student_name VARCHAR(30));
INSERT INTO club_b (student_name) VALUES ('Neha'), ('Pooja'), ('Meera');
```

1. Names in club_a that are also in club_b

```
SELECT student_name FROM club_a WHERE student_name IN (SELECT student_name FROM club_b);
```

2. Count names in club_b

```
SELECT COUNT(*) AS members FROM club_b;
```

3. Names in club_a, sorted

```
SELECT student_name FROM club_a ORDER BY student_name ASC;
```

7. Result

The SQL queries were executed and the results were checked against the requirements of the experiment.

8. Viva-Voce Questions

1. What is a subquery?
2. How is IN used with a subquery?
3. What does COUNT(*) return?

EXPERIMENT NO. 13

1. Experiment Title

UNION and INTERSECT

2. Aim

Combine two lists of names and find names present in both lists.

3. Software / Tools Required

MySQL or another compatible SQL database system, SQL editor/command-line client, and a tool to capture output screenshots.

4. Theory

This experiment practices the SQL concepts named in the experiment title. SQL statements are used to define database objects, manipulate records, retrieve data, and display query results according to the task requirements.

5. Algorithm / Procedure

1. Open the SQL editor and select the appropriate database.
2. Run the setup statements below, if provided, to create and populate the required tables.
3. Execute each task query in order.
4. Check the returned rows or database changes against the task requirements.
5. Capture the actual output for the practical record.

6. Queries

Setup (run first; not a task)

```
CREATE TABLE club_a (student_name VARCHAR(30));
INSERT INTO club_a (student_name) VALUES ('Asha'), ('Rohan'), ('Neha');
CREATE TABLE club_b (student_name VARCHAR(30));
INSERT INTO club_b (student_name) VALUES ('Neha'), ('Pooja'), ('Meera');
```

1. All names once, sorted (UNION)

```
SELECT student_name FROM club_a UNION SELECT student_name FROM club_b ORDER BY student_name;
```

2. Names present in both clubs (INTERSECT)

```
SELECT student_name FROM club_a INTERSECT SELECT student_name FROM club_b ORDER BY student_name;
```

3. All names including repeats (UNION ALL)

```
SELECT student_name FROM club_a UNION ALL SELECT student_name FROM club_b ORDER BY student_name;
```

7. Result

The SQL queries were executed and the results were checked against the requirements of the experiment.

8. Viva-Voce Questions

1. What is the difference between UNION and UNION ALL?
2. What does INTERSECT return?
3. Why must combined queries have compatible columns?

EXPERIMENT NO. 14

1. Experiment Title

Creating and Using a View

2. Aim

Save a query as a view, then read the view.

3. Software / Tools Required

MySQL or another compatible SQL database system, SQL editor/command-line client, and a tool to capture output screenshots.

4. Theory

This experiment practices the SQL concepts named in the experiment title. SQL statements are used to define database objects, manipulate records, retrieve data, and display query results according to the task requirements.

5. Algorithm / Procedure

1. Open the SQL editor and select the appropriate database.
2. Run the setup statements below, if provided, to create and populate the required tables.
3. Execute each task query in order.
4. Check the returned rows or database changes against the task requirements.
5. Capture the actual output for the practical record.

6. Queries

Setup (run first; not a task)

```
CREATE TABLE score (name VARCHAR(30), subject VARCHAR(20), marks INT);
INSERT INTO score (name, subject, marks) VALUES ('Asha', 'Maths', 78), ('Rohan', 'Maths', 64),
('Neha', 'Maths', 88), ('Kiran', 'Maths', 52);
```

1. Create the high_scores view

```
CREATE OR REPLACE VIEW high_scores AS SELECT name, marks FROM score WHERE marks >= 70;
```

2. Show all rows from the view

```
SELECT * FROM high_scores;
```

3. Show names in the view, sorted

```
SELECT name FROM high_scores ORDER BY name ASC;
```

7. Result

The SQL queries were executed and the results were checked against the requirements of the experiment.

8. Viva-Voce Questions

1. What is a database view?
2. How do you retrieve rows from a view?
3. What is one benefit of using a view?

EXPERIMENT NO. 15

1. Experiment Title

Indexes and Adding a Row

2. Aim

Create an index, read rows by city, and add a row.

3. Software / Tools Required

MySQL or another compatible SQL database system, SQL editor/command-line client, and a tool to capture output screenshots.

4. Theory

This experiment practices the SQL concepts named in the experiment title. SQL statements are used to define database objects, manipulate records, retrieve data, and display query results according to the task requirements.

5. Algorithm / Procedure

1. Open the SQL editor and select the appropriate database.
2. Run the setup statements below, if provided, to create and populate the required tables.
3. Execute each task query in order.
4. Check the returned rows or database changes against the task requirements.
5. Capture the actual output for the practical record.

6. Queries

Setup (run first; not a task)

```
CREATE TABLE customer (customer_id INT PRIMARY KEY, name VARCHAR(30) NOT NULL, city VARCHAR(20));
INSERT INTO customer (customer_id, name, city) VALUES (1, 'Kiran', 'Kochi'), (2, 'Deepak',
'Delhi'), (3, 'Lata', 'Mumbai'), (4, 'Asha', 'Delhi');
```

1. Create an index on city

```
CREATE INDEX idx_city ON customer (city);
```

2. Customers who live in Delhi

```
SELECT name FROM customer WHERE city = 'Delhi' ORDER BY name ASC;
```

3. Add a customer and show the table

```
INSERT INTO customer (customer_id, name, city) VALUES (5, 'Arjun', 'Pune');
SELECT * FROM customer ORDER BY customer_id ASC;
```

7. Result

The SQL queries were executed and the results were checked against the requirements of the experiment.

8. Viva-Voce Questions

1. What is an index?
2. How can an index help a query?
3. What is the purpose of a primary key?